S.No: 1	Exp. Name: ***Default Values of Primitive Datatypes using Static Keyword***	**Date: 2026-07-29**

Aim:

Write a Java program that demonstrates the default values of different data types.

When instance variables or static variables are declared without explicit initialization, Java automatically assigns default values.

Your task is to:

- Declare static variables of the following primitive data types (**byte**, **short**, **int**, **long**, **float**, **double**, **char**, **boolean**) and one reference type (**String**) without assigning any values.
- Print the default values of these variables.

Input Format:

- There is no input required for this program.

Output Format:

- The program should display the default values of all primitive data types and the **String** reference in the following format:

```
byte: <value>
short: <value>
int: <value>
long: <value>
float: <value>
double: <value>
char: [<value>]
boolean: <value>
String: <value>
```

Note: Print all static default values by adding their corresponding variable names after each concatenation operator (+).

Source Code:

Q69851/DefaultValues.java

```
package Q69851;
public class DefaultValues {
    static byte defaultByte;
    static short defaultShort;
    static int defaultInt;
        static long defaultLong;
    static float defaultFloat;
    static double defaultDouble;
    static char defaultChar;
    static boolean defaultBoolean;
    static String defaultString;

    public static void main(String[] args) {

        System.out.println("byte: " +defaultByte);
        System.out.println("short: " +defaultShort    );
        System.out.println("int: " +defaultInt);
        System.out.println("long: " +defaultLong);
        System.out.println("float: " +defaultFloat);
        System.out.println("double: " +defaultDouble);
        System.out.println("char: ["+defaultChar+"]");
        System.out.println("boolean: " +defaultBoolean);
        System.out.println("String: " +defaultString);
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
byte: 0
short: 0
int: 0
long: 0
float: 0.0
double: 0.0
char: []
boolean: false
String: null

S.No: 2	Exp. Name: ***Roots of the quadratic equation***	**Date: 2026-08-12**

Aim:

Write a Java Program to find **Roots** of a Quadratic Equation.

Refer to the displayed sample test cases to strictly match the input and output layout.

Source Code:

q27331/QuadraticEquation.java

```
package q27331;
import java .util.Scanner;
class QuadraticEquation
        {
                public static void main(String args[]){
                        Scanner s =new Scanner(System.in);
                                System.out.print("Coefficient a: ");
                        float a= s.nextFloat();
                        System.out.print("Coefficient b: ");
                        float b= s.nextFloat();
                        System.out.print("Coefficient c: ");
                        float c = s.nextFloat();
                        float D = b*b-(4*a*c);
                        if(D==0){
                                System.out.println("The roots are real
and equal");
                                System.out.println("Root: "+(-b/2*a));
                        }
                        else if(D<0)
                                System.out.println("The roots are
imaginary");
                        else{
                                System.out.println("The roots are real
and distinct");
                                System.out.println("Root1: "+((-b+
(Math.sqrt(D)))/(2*a))+" Root2: "+((-b-(Math.sqrt(D)))/(2*a)));
                        }
                }
        }
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Coefficient a:
1
Coefficient b:
6
Coefficient c:
9
The roots are real and equal
Root: -3.0

Test Case - 2
User Output
Coefficient a:
1
Coefficient b:
5
Coefficient c:
8
The roots are imaginary

Test Case - 3
User Output
Coefficient a:
2
Coefficient b:
6
Coefficient c:
1
The roots are real and distinct
Root1: -0.17712434446770464 Root2: -2.8228756555322954

Test Case - 4

Coefficient a:
2
Coefficient b:
6
Coefficient c:
4
The roots are real and distinct
Root1: -1.0 Root2: -2.0

S.No: 3	Exp. Name: ***Binary Search***	Date: 2026-08-19

Aim:

Write a program to check whether a given element is present or not in a sorted array using the Binary Search technique.

Input Format:

- The first line of input contains an integer N, representing the number of elements in the array.
- The second line of input contains N space-separated integers representing the elements of the array.
- The last line of input contains an integer representing the element to be searched.

Output Format:

- If the element is found, print the index.
- If the element is not found, print "Not found".

Sample Test Case:

Input:

```
7
1 2 3 3 4 5 6
3
```

Output:

```
2
```

Note:

- If duplicate elements are present, print the index of the first occurrence.
- The array is always given in sorted order.
- Refer to the visible test cases for better understanding.

Source Code:

q17128/CTJ17128.java

```
package q17128;
import java.util.Scanner;
public class CTJ17128 {
        public static void main(String args[])
        {
                int [] arr;
                arr=new int[100];
                Scanner s = new Scanner(System.in);
                int n= s.nextInt();
                int low=0;
                int high =n-1;
                int j=0,count=0;
                for(int i=0;i<n;i++)
                        arr[i]=s.nextInt();
                int key= s.nextInt();
                while(low<=high){
                        int mid=(low+high)/2;
                        if(arr[mid]==key){
                                for(j=0;j<mid;j++){
                                        if(arr[j]==key){
                                                count++;
                                                if(count==1)

System.out.println(j);
                                }
                                }
                                        if(count==0)

System.out.println(mid);
                                        break;
                        }
                        else if(key<arr[mid])
                                high=mid-1;
                        else
                                low=mid+1;
                }
                if (low>high)
                System.out.println("Not found");
                }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
7
1 2 3 3 4 5 6
3
2

Test Case - 2
User Output
10
1 2 3 4 5 6 7 8 9 19
20
Not found

S.No: 4	Exp. Name: ***Bubble Sort Technique***	**Date: 2026-08-19**

Aim:

Write a Java program to sort an array of integers using the bubble sort algorithm. The program should read the number of elements and the array elements from the user, then display the sorted array in ascending order.

Input Format:

- The first line contains an integer n representing the number of elements in the array.
- The second line contains n space-separated integers representing the array elements.

Output Format:

- Print the sorted array elements in ascending order, separated by a space:

```
<int1> <int2> <int3> ...
```

Note:

- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q32083/BubbleSort.java

```
package q32083;
import java.util.Scanner;

public class BubbleSort {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        bubbleSort(arr);

        for (int num : arr) {
            System.out.print(num + " ");
        }

        scanner.close();
    }

    public static void bubbleSort(int[] arr) {
                int i,j;
        for(i=0;i<arr.length;i++){
                        for(j=i+1;j<arr.length;j++){
                                if(arr[i]>arr[j]){
                                        int temp=arr[j];
                                                arr[j]=arr[i];
                                                arr[i]=temp;
                                }
                        }
                }




        //Type your content here
```

```
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
12 35 69 48 51
12 35 48 51 69

Test Case - 2
User Output
3
5 -1 25
-1 5 25

S.No: 5	Exp. Name: ***Removing duplicate characters from a glitchy message by using StringBuffer constructor***	**Date: 2026-08-19**

Aim:

A group of friends sent a message to Jaya, but due to a technical glitch, the message contained duplicate characters. To overcome this kind of problem she decided to craft a Java program using a **StringBuffer** constructor that takes the glitchy message as input and returns a new string with all duplicate characters removed.

How can Jaya craft this program?

Input Format:

The input line reads a string representing the glitchy message.

Output Format:

The output is a string after removing the duplicate characters (Including special characters, space).

Note: Use println() to print the output statements.

Source Code:

q22135/Main.java

```
package q22135;
import java.util.Scanner;
// write your code here
public class Main{
        public static void main(String args[]){
                Scanner s = new Scanner(System.in);
                String input = s.nextLine();
                StringBuffer result = new StringBuffer();
                for(int i=0;i<input.length();i++){
                        char ch = input.charAt(i);
                if(result.indexOf(String.valueOf(ch))==-1){
                        result.append(ch);
                }
                }
                System.out.println(result);
        s.close();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
abcdef
abcdef

Test Case - 2
User Output
a a b b c c
a bc

S.No: 6	Exp. Name: ***Constructor Overloading***	**Date: 2026-08-19**

Aim:

Write a Java program that demonstrates constructor overloading in a class named **Book**. The **Book** class should have the following attributes:

- **title** - String
- **author** - String
- **pageCount** - int

Implement the following constructors in the **Book** class:

1. A default constructor that initializes the title and author to "Unknown Title" and "Unknown Author," respectively, and sets **pageCount** to 0.
2. A constructor that takes **title** and **author** as parameters and initializes the corresponding attributes. The **pageCount** should be set to 0.
3. A constructor that takes **title, author**, and **pageCount** as parameters and initializes the corresponding attributes.

Additionally, include a method named **displayBookDetails** that prints the details of the book, including the **title, author,** and **pageCount.**

Sample Input and Output format:

```
Details for Book 1:
Title: Introduction to Java Programming
Author: John Smith
Details for Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson
Page Count: 350
Book 1:
Title: Introduction to Java Programming
Author: John Smith
Page Count: 0
Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson
Page Count: 350
```

Source Code:

Book.java

```
import java.util.Scanner;

public class Book {
String title;
String author;
int pageCount;
Book(String title,String author){
        this.title=title;
        this.author=author;
        this.pageCount=0;
}
Book(String title,String author,int pageCount){
        this.title=title;
        this.author=author;
        this.pageCount=pageCount;
}
void displayBookDetails(){
        System.out.println("Title: "+title);
        System.out.println("Author: "+author);
        System.out.println("Page Count: "+pageCount);
}


     public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking user inputs for Book 1
        System.out.println("Details for Book 1:");
        System.out.print("Title: ");
        String title1 = scanner.nextLine();
        System.out.print("Author: ");
        String author1 = scanner.nextLine();

        // Creating Book 1 using the constructor with title and author
parameters
        Book book1 = new Book(title1, author1);

        // Taking user inputs for Book 2
        System.out.println("Details for Book 2:");
        System.out.print("Title: ");
        String title2 = scanner.nextLine();
        System.out.print("Author: ");
        String author2 = scanner.nextLine();
        System.out.print("Page Count: ");
```

```
        int pageCount2 = scanner.nextInt();

        // Creating Book 2 using the constructor with all parameters
        Book book2 = new Book(title2, author2, pageCount2);

        // Displaying details of each book
        System.out.println("Book 1:");
        book1.displayBookDetails();

        System.out.println("Book 2:");
        book2.displayBookDetails();

        // Close the scanner
        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Details for Book 1:
Title:
Introduction to Java Programming
Author:
John Smith
Details for Book 2:
Title:
Data Structures and Algorithms
Author:
Alice Johnson
Page Count:
350
Book 1:
Title: Introduction to Java Programming
Author: John Smith
Page Count: 0
Book 2:
Title: Data Structures and Algorithms
Author: Alice Johnson

ID: 25B01A12F9
2025-2029-IT-C
Shri Vishnu Engineering College for Women (Autonomous)

Page Count: 350

Test Case - 2
User Output
Details for Book 1:
Title:
Programming Basics
Author:
Jane Doe
Details for Book 2:
Title:
Advanced Java Concepts
Author:
Bob Williams
Page Count:
500
Book 1:
Title: Programming Basics
Author: Jane Doe
Page Count: 0
Book 2:
Title: Advanced Java Concepts
Author: Bob Williams
Page Count: 500

S.No: 7	Exp. Name: ***Implementing Method overloading***	**Date: 2026-10-07**

Aim:

Write a Java program with a class name OverloadArea with overload methods area(float) and area(float, float) to find area of square and rectangle.

Write the main method within the class and assume that it will receive a total of 2 command line arguments of type float.

If the main() is provided with arguments : 1.34, 1.98 then the program should print the output as:
Area of square for side in meters 1.34 : 1.7956
Area of rectangle for length and breadth in meters 1.34, 1.98 : 2.6532001

Source Code:

OverloadArea.java

```
class OverloadArea{
        float area(float s){
                return s*s;
        }
        float area(float l,float b){
                return l*b;
        }
        public static void main(String args[]){
                OverloadArea a=new OverloadArea();
                float s=Float.parseFloat(args[0]);
                float l=s;
                float b = Float.parseFloat(args[1]);
                System.out.printf("Area of square for side in meters
%.2f: %.4f\n",s,a.area(s));
                System.out.printf("Area of rectangle for length and
breadth in meters %.2f, %.2f: %.7f\n",l,b,a.area(l,b));
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Area of square for side in meters 1.34: 1.7956
Area of rectangle for length and breadth in meters 1.34, 1.98: 2.6532001

Test Case - 2
User Output
Area of square for side in meters 2.30: 5.2900
Area of rectangle for length and breadth in meters 2.30, 2.80: 6.4399996

S.No: 8	Exp. Name: ***Basics of Class and Object***	**Date: 2026-10-07**

Aim:

The Spy Academy is implementing a new system to manage information about their agents. As part of this system, they need to create a class to represent an agent. Imagine yourself as the chief developer, and your task is to design a basic class named **Agent**.

Task:

1. Design a class named **Agent** with the following attributes:
   - **name** (String)
   - **age** (int)
   - **codename** (String)
   - **isUndercover** (boolean)
2. Implement a method named **introduce()** that prints a message introducing the agent, including their **name**, **age**, and **codename**.

Additional Instructions:

- In the main method, create an instance of the **Agent** class named **topAgent**.
- Use user input within the main method to set the attributes of **topAgent** (**name**, **age**, **codename**, and **isUndercover**).
- Invoke the **introduce()** method on **topAgent** to introduce this top agent to the team.

Note:

- Partial code is given fill in the remaining code.

Source Code:

q22852/SpySystem.java

```
package q22852;
import java.util.Scanner;

// create class Agent and Attributes
class  Agent{
        String name;
        int age;
        String codename;
        boolean isUndercover;



    // create Method to introduce the agent

    void  introduce(){
        System.out.println("Hello, I am Agent " +codename+ "."); //
write your code here
        System.out.println("My real name is " +name+ "."); // write
your code here
        System.out.println("I am " +age+ " years old."); // write your
code here
    }
}

public class SpySystem {
    public static void main(String[] args) {
        // Create an instance of Agent
        Agent topAgent=new Agent();
        Scanner scanner = new Scanner(System.in);
        System.out.print("name: ");
        topAgent.name = scanner.nextLine();
        System.out.print("age: ");
        topAgent.age = scanner.nextInt();
        scanner.nextLine();
        System.out.print("codename: ");
        topAgent.codename = scanner.nextLine();
        System.out.print("undercover? (true/false): ");
        topAgent.isUndercover = scanner.nextBoolean();

        topAgent.introduce();
        // Introducing the topAgent

        scanner.close();
```

```
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
name:
Alice
age:
27
codename:
007
undercover? (true/false):
true
Hello, I am Agent 007.
My real name is Alice.
I am 27 years old.

Test Case - 2
User Output
name:
Bob
age:
25
codename:
117
undercover? (true/false):
false
Hello, I am Agent 117.
My real name is Bob.
I am 25 years old.

S.No: 9	Exp. Name: ***Constructor Overloading and Default Constructor***	**Date: 2026-10-07**

Aim:

You are given a Java program that demonstrates the concept of constructor overloading.

The class **Test** contains:

1. Two private integer variables i and j.
2. Three overloaded constructors:
   - A **default constructor** that sets both i and j to 0.
   - A **constructor with one parameter** that sets i to the given value and j to 0.
   - A **constructor with two parameters** that sets both i and j using the given values.
   - Implement a method **display()** that prints the values of i and j.
3. The program creates three objects of the **Test** class
   - **obj1** - created using the default constructor.
   - **obj2** - created using the constructor with one input value for i.
   - **obj3** - created using the constructor with two input values for i and j.

Input Format:

- First, **obj1** uses the default constructor - no input.
- After displaying **obj1**, read an integer from the user - this is used for i in **obj2**.
- After displaying **obj2**, read two integers - these are used for i and j in **obj3**.

Output Format:

- The program displays the values of i and j for each object in the following format:

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:<i_value>
Displaying values for obj2:
Value of i: <value>
Value of j: 0
Enter a value for i:<i_value>
Enter a value for j:<j_value>
Displaying values for obj3:
Value of i: <i_value>
Value of j: <j_value>
```

Sample Test Case (Combined Input and Output):

```
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i: 10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i: 40
Enter a value for j: 50
Displaying values for obj3:
Value of i: 40
Value of j: 50
```

Note:

- Input and output are combined in the test cases. Please ensure your program takes input and displays output simultaneously, in the same sequence as shown in the sample interaction.
- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

q21947/Main.java

```
package q21947;
import java.util.Scanner;
class Test {
    // Private instance variables
    private int i;
    private int j;

    // Default constructor
    Test(){
                i=0;
                j=0;
        }
    Test(int a){
                j=0;
                i=a;
        }
    Test(int a,int b){
                i=a;
                j=b;
        }
    // Parameterized constructor with one parameter



    // Parameterized constructor with two parameters



    // Display function to print the values of i and j

        void display(){
                System.out.println("Value of i: "+i);
                System.out.println("Value of j: "+j);
        }
}
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Test obj1 = new Test();
        System.out.println("Displaying values for obj1:");
        obj1.display();
        System.out.print("Enter a value for i: ");
        int value1 = scanner.nextInt();
        Test obj2 = new Test(value1);
```

```
        System.out.println("Displaying values for obj2:");
        obj2.display();
        System.out.print("Enter a value for i: ");
        int value2 = scanner.nextInt();
        System.out.print("Enter a value for j: ");
        int value3 = scanner.nextInt();
        Test obj3 = new Test(value2, value3);
        System.out.println("Displaying values for obj3:");
        obj3.display();
        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i:
40
Enter a value for j:
50
Displaying values for obj3:
Value of i: 40
Value of j: 50

Test Case - 2
User Output
Displaying values for obj1:
Value of i: 0
Value of j: 0

```
Enter a value for i:
10
Displaying values for obj2:
Value of i: 10
Value of j: 0
Enter a value for i:
20
Enter a value for j:
30
Displaying values for obj3:
Value of i: 20
Value of j: 30
```

S.No: 10	Exp. Name: ***Number and Square - Single Inheritance***	**Date: 2026-10-07**

Aim:

Write a Java program to demonstrate **single inheritance** by reading an integer and displaying its square.

Create a class named **NumberIn** with the following members:

- An integer data member **num**.
- A method **inputNum()** that reads an integer from the user and stores it in **num**. The method should display the prompt:

```
Enter number:
```

Create another class named **SquareOut** that extends **NumberIn** and implements the following member:

- A method **displaySquare()** that displays the square of the entered number.

Note: The **MainFunction** class has already been provided in the editor. It creates an object of the **SquareOut** class, invokes the **inputNum()** method to read the input, and then invokes the **displaySquare()** method to display the result. Do not modify the **MainFunction** class. Implement only the **NumberIn** and **SquareOut** classes.

Source Code: